

Software factories: evidence for two product visions

28 September 2026 • 21 Sources

EXECUTIVE SUMMARY

The useful opportunity for herdr-graph is a durable organizational foundation beneath a configurable factory. The surveyed systems emphasize different layers: Gas Town provides a coordination organization; StrongDM describes a factory operating philosophy; Attractor specifies workflow execution; Symphony coordinates issue-driven runs; OpenHands supplies an agent substrate; Devin packages agents and reusable working practices. These are complementary comparison points rather than interchangeable implementations. [1][5][7][8][10][12]

The local design gives graph a distinct purpose: persistent teamspaces, seats, and clones in Herdr; organizational instructions and relationships; and recoverable state changes. The concrete factory adds foreman-led project teams, temporary feature collaborators, knowledge stewardship, and a chosen delivery practice. This division preserves the user's existing decisions while allowing other organizations to use the foundation differently.

Operator accounts support the appeal of delegated coordination but reveal adoption costs. Early Gas Town users describe productive parallel work alongside observability, recovery, and mental-model problems. Wheelhouse's creator reports both useful output and serious resource and policy-complexity constraints. StrongDM-related discussions show interest in scenario validation, with continuing concerns about code quality and maintainability. These are qualitative signals, not a representative survey or comparative performance study. [2][14][15][16][18][19][20]

The resulting recommendation is to lead both visions with continuity, intelligible ownership, and accepted outcomes. Keep staffing flexible, distinguish completion claims from evidence, and make recovery part of the ordinary experience. Treat cost and human attention as product concerns. Avoid equating agent count, commit volume, or a closed work item with value. The strongest evidence supports specific documented mechanisms and reported experiences; it does not establish that any factory is universally reliable, economical, or suitable for unattended production.

INTRODUCTION

The research question is: which principles and operator lessons should inform two products—herdr-graph as a Herdr-native foundation and a concrete software factory built on it? The user clarified that the outputs are product visions rather than whitepapers or implementation specifications. The two primary deliverables are [the foundation vision](#) and [the factory vision](#).

Local authority comes from [DESIGN-NOTES.md](#), [RESUME.md](#), and a directly acknowledged [consultation with the adjacent designer](#). External projects supply examples and challenges; they do not override those decisions. In particular, the foundation has no fixed organizational hierarchy or model wake scheduler, and the concrete factory has not been authorized as an unattended deployment system.

This is a targeted qualitative study. Official documentation establishes what projects describe or intend; first-person accounts establish what their authors report experiencing. No surveyed factory was installed or benchmarked. Mutable documentation was retrieved on the research date, without pinning all repositories to commits. Wheelhouse is accessible here through its creator's accounts, not an independent implementation audit. [3]

MAIN ANALYSIS

Finding 1: compare layers before comparing factories

Gas Town's public documentation makes a coordinator, workers, supervision, durable work, and integration visible parts of its organization. That is useful evidence for assigning ownership to coordination work. It is not an argument that every organization requires the same named roles. [1]

StrongDM's first-party factory account places specifications and scenario validation at the center of development. Its no-human-code-review position is an operating choice. Attractor, separately, specifies a workflow engine with checkpoints, extensible handlers, conditional routing, and human decision points. One team's philosophy and its reusable mechanisms do not have identical boundaries. [5][6][7]

Symphony describes an issue-driven coordination service, with policy in a repository workflow and operational mechanisms for bounded execution and recovery. The specification explicitly limits its scope rather than presenting a complete enduring organization. OpenHands likewise provides agent capabilities and application surfaces; its SDK is not itself a factory rulebook. [8][9][10]

This comparison changes the initial research framing. Asking which factory to emulate obscures the more useful question: which layer owns a behavior? Graph should own durable organizational state and its lifecycle. A factory should own its professions, acceptance policy, and working habits. Execution engines can support either without becoming the definition of a seat. This is a design inference corroborated by the separation of mechanisms and policy in Attractor, Symphony, and OpenHands—not independent validation of graph's exact proposed architecture. [7][9][11]

Finding 2: continuity is a valuable product boundary

The local design's seat/clone/session distinction answers a problem that a workflow alone does not: who remains responsible after a run finishes? Herdr supplies the addressable workspace, tab, and pane surfaces; graph gives those surfaces organizational meaning through its adopted mapping. Herdr's own documentation does not claim that its tabs are durable organizational seats. [4]

Yegge's September discussion presents seats as responsibilities with inherited context, while his August account argues strongly for application-specific harnesses. The productive tension is between persistent organizational identity and tailored operating practice. A reusable foundation can address the former while leaving the latter to the instance; whether that boundary proves useful remains a product hypothesis. [2][3]

Devin's playbooks distinguish reusable procedures from broader organizational knowledge. Its dynamic workflows describe resumable background execution that leaves the coordinating conversation available. This is a relevant experience comparison for the user's wish to discuss a responsibility while another clone works, but it does not imply equivalent clone identity or shared-state semantics. [12][13]

The proposed product should therefore make continuity tangible through ordinary actions: load a responsibility, discover its commitments, open a side conversation, reassign work, and retire or resurrect a seat. A compelling demonstration would show useful recovery across occupant replacement rather than simply launching many agents. That recommendation comes primarily from the local design, with external examples helping define the contrast.

Finding 3: sentiment favors delegation when progress remains understandable

Three early Gas Town trials provide a mixed picture. Tim Sehn liked the Mayor interface but reported no acceptable result from four attempted fixes. Tenzin Wangdhen reported six merged PRs from seven tasks during one dinner, yet ultimately preferred borrowing ideas over adopting the whole system. Justin Abrahms described completing work and recovering after a restart while finding the vocabulary and startup model difficult to understand. These are different workloads and authors, not a shared benchmark. [14][15][16]

Wangdhen's complaints about observability and prodding are especially relevant to product design. His historical orphan-process report explicitly says the issue was later fixed, so it should not be repeated as a current defect. Sehn's roughly \$100 trial cost likewise belongs to that experiment and billing context, not to a universal price for Gas Town. Two of the three authors disclose knowing Yegge; this limits any claim of fully independent sampling. [14][15][16]

A separate issue report describes concurrent dispatch returning success while assignments failed to persist. Retrieved snapshots disagree about its later resolution state. The defensible use is to illustrate a historical class of discrepancy between an operation response and durable state, without asserting that current versions have the bug. [17]

Our inference is that user trust depends on inspectable ownership, progress, and exceptions. Graph's intended durable mutation journal and reconciliation fit that concern, but persistence alone is insufficient: the human needs to understand whether a request was admitted, committed, applied, or failed. The factory also needs to explain whether a worker finished, checks passed, an owner accepted the result, or a release occurred. Neither product should promise those states are equivalent.

Finding 4: evidence of behavior does not eliminate stewardship

StrongDM's scenarios and behavioral service twins put validation of outcomes ahead of generated implementation volume. That is a useful challenge to factories organized around task closure alone. It remains a first-party report, and adopting its validation emphasis does not entail adopting every policy in its operating model. [\[5\]\[6\]](#)

The associated Hacker News discussion includes actual Attractor experiments as well as skeptical code inspection. Favorable accounts describe small applications or incomplete stacks; they do not demonstrate production economics. Criticisms include code quality concerns. A former customer's objection was corrected in-thread to distinguish the research lab from the core product; repeating that objection without its correction would misrepresent the evidence. [\[18\]](#)

Another operator describes a positive Attractor fork experience using property tests, fault injection, fuzzing, browser checks, and scenarios. The account provides no controlled outcomes or cost record. OctopusGarden's creator similarly reports functional CRUD/REST results alongside difficulty maintaining or repairing the generated code. These observations suggest evaluating maintainability and recoverability as well as behavioral success. [\[19\]\[20\]](#)

The proposed factory should make acceptance an explicit project practice. Some projects may delegate it extensively; others may rely on direct human assessment. Graph should preserve the attribution and links needed to tell that story, without deciding what every project must accept. This is a recommendation, not a finding that any particular review policy is universally superior.

Finding 5: organizational overhead needs an owner

Yegge's Wheelhouse accounts caution that successful generation can coexist with costly operation and accumulated restrictions. These are creator reports about a private system, not audited measurements. They are nevertheless useful counterevidence to the assumption that more roles, more rules, or more output necessarily improves the human's experience. [\[2\]\[3\]](#)

The official landscape also offers narrower approaches. Symphony constrains its job to coordination and execution. OpenHands' design discusses separating application logic from the agent core. The SWE-agent repository recommends a simpler successor. None establishes a universal optimum, but together they justify questioning machinery that does not earn its place. [\[9\]\[11\]\[21\]](#)

For graph, the proposed implication is cheap dormant responsibility and ordinary non-model bookkeeping. For the factory, it is deliberate staffing, attention to unfinished work, and explicit ownership of rule maintenance. A useful rule needs a reason to exist; an old incident need not create a permanent layer of bureaucracy. These choices align with the user's peer-trust stance.

Factory improvement also needs to remain distinguishable from product work. It may be worthwhile, but it should not quietly consume the project it exists to serve. Release pacing, review capacity, and the human's attention can constrain useful delivery even when coding is fast. Proposed success measures should therefore follow accepted outcomes, rework, recovery effort, and whole-system cost rather than maximizing activity.

SYNTHESIS & INSIGHTS

The two-product split is a defensible design hypothesis because it separates two kinds of change. Identity, history, participation, and recoverable organizational state should remain stable across a change in operating practice. Roles, escalation, review, and delivery methods need to vary by project and evolve through use. The surveyed architectures show several ways to separate mechanisms from policy, but no source proves that the proposed graph primitives cover every factory. [7][9][11]

The user-facing distinction is equally useful. Graph answers where responsibility lives and how it continues. The factory answers how this particular team turns an intention into a useful result. A product vision should make that experience legible before describing every queue, adapter, or schema. The detailed design can then test whether the intended behavior survives concurrency, failure, and manual intervention.

The suggested first proving ground is the user's foreman-led project team. A differently organized second instance would then challenge accidental assumptions. Broad generalization should be earned through those differences; it should not begin as a promise of drop-in compatibility with Gas Town, Wheelhouse, or any other engine.

COUNTEREVIDENCE REGISTER

Initial attraction	Counterevidence or limit	Consequence for the visions
More agents create more throughput	Operator accounts report attention and observability costs [14][15][16]	Staffing must improve outcomes, not just activity
A common factory can suit every project	Wheelhouse's author advocates bespoke harnesses [3]	Reuse organizational mechanisms; keep practice configurable
Passing scenarios settles quality	Creator/operator accounts still describe maintenance concerns [18][20]	Acceptance includes the project's ownership requirements
Persistent state ensures delivery	Historical dispatch issue separates success response from persistence [17]	Show admission, application, uncertainty, and recovery distinctly
More rules improve reliability	Wheelhouse creator reports policy overgrowth [2]	Make amendment and simplification part of instance practice
Positive online reaction establishes maturity	Favorable examples include toy apps and unverified self-reports [18][19]	Treat sentiment as discovery input, not proof

CLAIMS-EVIDENCE TABLE

Claim	Support	Status
Mechanism/policy separation recurs across projects	Attractor, Symphony, OpenHands [7][9][11]	Cross-project synthesis; not a benchmark
Early Gas Town trials combine appeal and friction	Sehn, Wangdhen, Abrahms [14][15][16]	Multiple first-person accounts; self-selected sample
Wheelhouse motivates seats but raises cost/complexity questions	Yegge [2][3]	One creator cluster; private implementation
Scenario evaluation can coexist with human decision points	StrongDM philosophy and Attractor spec [5][7]	Documentary distinction; same authoring organization
Herdr provides the native UI surfaces used in the local mapping	Herdr concepts and local design [4]	Documented surface; graph semantics remain proposed
Graph plus a configurable factory is the right product direction	Local decisions plus synthesis above	Product recommendation, to be tested through use

LIMITATIONS & CAVEATS

The research does not rank products or estimate comparative costs, reliability, security, or productivity. Official pages are primary evidence of documented intent, not independent proof of performance. Several sources share an author or organization and must not be counted as independent corroboration. Project-specific claims often have one primary source; their status remains explicit rather than padded with retellings.

Sentiment is English-language, self-selected, and weighted toward technically engaged early adopters. Hacker News identities and outcomes were not independently verified. A favorable comment whose authorship was disputed was excluded from human sentiment evidence. January and February Gas Town trials cannot establish September behavior; the Wheelhouse material concerns a different private system. Source dates and unresolved cache discrepancies are retained in the ledgers.

The existing local Herdr identity audit remains the relevant installed-version qualification. Current public documentation should not silently replace that evidence. The threads extension is designed and handed off, not verified implemented. The latest adjacent design approves current-intent reconciliation and explicit reassignment and cancellation of unresolved graph requests. Graph's journal concerns organizational mutations; project tasks and knowledge remain in their existing systems.

RECOMMENDATIONS

Use the foundation vision to articulate continuity, discoverability, composition, and recoverable operation in native Herdr. Keep its universal vocabulary small. Its first proof should be a responsibility that survives conversation replacement and partial failure while staying intelligible to the human.

Use the factory vision to propose a concrete working relationship: a standing foreman-led team, temporary feature collaborators, compact orientation, explicit acceptance, and practical peer trust. Make its templates useful defaults rather than staffing limits. Keep additional office functions, scheduling policies, budgets, and delegated release practices visibly optional or still to be designed.

Evaluate through real project outcomes before promising broad compatibility or large-scale autonomy. Measure what the human receives and what it costs to obtain and maintain it. Preserve contradictory observations; they are material for improving the product boundary rather than obstacles to a positive narrative.

METHODOLOGY APPENDIX

Used the requested deep-research skill, whose existing default-profile copy matched a freshly downloaded upstream package. Installed identical copies into the three AISW Codex profiles; hashes and destinations are in `skill-installation.json`. Read its methodology and quality guidance, checked the system date, and initialized a persisted research run. Search CLI was used first; sandbox-blocked requests were retried with authorization. Browser retrieval checked source pages. Two delegated research tasks covered the official landscape and operator sentiment.

Evidence was saved before synthesis in `sources.jsonl`, `evidence.jsonl`, `claims.jsonl`, and the companion landscape/sentiment ledgers. Source-specific provenance and caveats remain in those packets. The outline shifted from a survey of interchangeable factories to five findings about layers, continuity, legibility, acceptance, and overhead. The reason was documentary evidence that the surveyed projects operate at different layers, plus user clarification that the deliverables are product visions.

Self-review used three questions: would an operator recognize the account; would a skeptical reviewer distinguish self-report from validation; and could an implementer tell settled direction from a new proposal? Initial revisions retained the designer's retry decision and removed any implication of universally compatible engines or proven threads integration. A completion audit incorporated subsequent approval of latest-intent reconciliation, request reassignment and cancellation, and the explicit limit of graph state to organizational concerns. Structure and citation checks accompany the report. Link verification cannot establish that a source is true; substantive support was checked during retrieval and synthesis.

The package is project-local to support the requested vision documents. It uses a compact synthesis rather than the skill's longer per-finding word targets. HTML and PDF are reading copies; Markdown and evidence ledgers remain authoritative. No application code, live organization, or deployment was created, and the designer's source notes were left intact.

BIBLIOGRAPHY

- [1] Gas Town maintainers. "Gas Town README". [Project documentation](#).
- [2] Steve Yegge (2026). "Seats and Sunsets". September 15. [Creator account](#).
- [3] Steve Yegge (2026). "The Shape of Things to Come". August. [Creator account](#).
- [4] Herdr. "Concepts". [Official documentation](#).
- [5] StrongDM (2026). "Software Factories And The Agentic Moment". February 6. [First-party account](#).

- [6] StrongDM. "Principles". [Factory principles](#).
- [7] StrongDM. "Attractor Specification". [Specification](#).
- [8] OpenAI. "Symphony README". [Project documentation](#).
- [9] OpenAI. "Symphony Service Specification". [Specification](#).
- [10] OpenHands. "Software Agent SDK". [Official documentation](#).
- [11] OpenHands. "Design Principles". [Architecture documentation](#).
- [12] Cognition. "Creating Devin Playbooks". [Official documentation](#).
- [13] Cognition. "Devin Dynamic Workflows". [Official documentation](#).
- [14] Tim Sehn (2026). "A Day in Gas Town". January 15. [Operator report](#).
- [15] Tenzin Wangdhen (2026). "Gas Town: The Good, The Bad, The Ugly". February 19. [Operator report](#).
- [16] Justin Abrahms (2026). "Gas Town: Running Dozens of Claude Code Instances at Once". January 5. [Operator report](#).
- [17] patrickclancy (2026). "Gas Town issue 3114". March 21. [Historical issue report](#).
- [18] Hacker News participants (2026). "Software factories and the agentic moment discussion". February. [Discussion](#).
- [19] MarkMarine (2026). "Attractor operator reply on Symphony discussion". March; exact day unverified. [First-person comment](#).
- [20] foundatron (2026). "Show HN: OctopusGarden". March; exact day unverified. [Creator report](#).
- [21] SWE-agent maintainers. "SWE-agent README". [Project documentation](#).